Lecture 1

Tuesday, August 25, 2026 12:26 PM

CS 6515: Graduate Algorithms

4540

Lecture 1: Course Overview, Asymptotic Notation, and Merge Sort

Instructors: Sahil Singla and Gerandy Brito

Fall 2026

1 Announcements

1. PhD students on waitlist should email Sahil (ssingla@gatech.edu) their gtid, email, and PhD program name. ← Add Section A in email
2. HW1 will be released by tomorrow. It will be due Sep 3.

2 What is this course about?

An *algorithm* is a sequence of instructions that accomplishes a task. We usually want more than correctness: we also want to minimize the resources used, such as time, space, or communication.

1. There is no single idea that solves every algorithmic problem. Instead, we will develop general approaches that apply to broad classes of problems;
2. learn useful abstractions and mathematical/proof techniques;
3. develop a theoretical way of thinking (a.k.a. theoretical lens) about new questions: useful even beyond computer science in fields, e.g., in fields such as economics, mathematics, and physics.

During the course, you are encouraged to use AI to learn and self-evaluate. The goal is to be curious, always learning, and develop skills to learn new things quickly.

Course roadmap

Part I Divide and conquer; dynamic programming; graph algorithms.

Part II Linear programming; convex optimization and gradient descent; approximation algorithms.

Part III Randomized algorithms; complexity theory.

Most communication, homework, and class notes will be posted on Piazza. Please consult the current syllabus on Canvas for grading policies, office hours, and other administrative details.

3 Asymptotic notation

For an input of size n, let $T(n)$ (or more generally $f(n)$) denote the amount of a resource used by an algorithm. We typically count *elementary operations* (like addition, subtraction, multiplication, and bit operations) and focus on how this quantity grows as the integer n becomes large.

5eee

3.1 Big-$\mathcal{O}$ notation: asymptotic upper bounds

Definition (Big-$\mathcal{O}$). Let $f, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$. We write

$$f(n) = \mathcal{O}(g(n))$$

if there exist constants $c > 0$ and $n_0 \in \mathbb{N}$ such that

$$f(n) \leq c\, g(n) \qquad \text{for every } n \geq n_0.$$

Thus, g is an eventual upper bound on f, up to a constant factor. When the limit exists, a useful test is

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} < \infty \quad \Longrightarrow \quad f(n) = \mathcal{O}(g(n)).$$

Example. We have $n^2 + 2n = \mathcal{O}(n^2)$: for $n \geq 1$, $n^2 + 2n \leq 3n^2$. We also have $n^2 + 2n = \mathcal{O}(n^3)$. Big-$\mathcal{O}$ is an upper bound, not necessarily the tightest upper bound.

3.2 Lower bounds, strict bounds, and tight bounds

Definition (Big-Ω). We write $f(n) = \Omega(g(n))$ if $g(n) = \mathcal{O}(f(n))$. Equivalently, there are constants $c > 0$ and n_0 such that $f(n) \geq c\, g(n)$ for every $n \geq n_0$.

Definition (Little-o and Little-ω). We write $f(n) = o(g(n))$ if, for every constant $c > 0$, there is an n_0 such that $f(n) < c\, g(n)$ for every $n \geq n_0$. For positive g, this is equivalent to

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0.$$

Intuitively, f grows strictly more slowly than g.

Similarly, $f(n) = \omega(g(n))$ means $g(n) = o(f(n))$.

Definition (Big-Θ). We write $f(n) = \Theta(g(n))$ if both $f(n) = \mathcal{O}(g(n))$ and $f(n) = \Omega(g(n))$.

Some useful comparisons are

$$1 = o(\log\log n) = o(\log n) = o(n^{0.1}) = o(n) = o(2^n) = o(2^{n^2}).$$

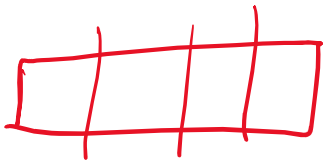
Also, $n! = o(n^n)$, since

$$\frac{n^n}{n!} = \prod_{i=1}^{n} \frac{n}{i} \longrightarrow \infty.$$

Exercise. Prove that

$$\log(n!) = \Theta(n \log n).$$

One route is to compare $n!$ with n^n for the upper bound and to use only the largest $n/2$ factors in the product for the lower bound.



4 Divide and Conquer

The divide-and-conquer paradigm has three steps:

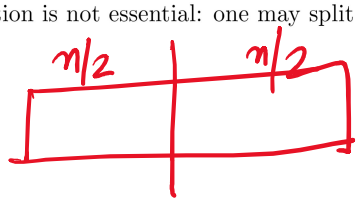
1. **Divide:** break the problem into smaller subproblems.
2. **Conquer:** solve each subproblem recursively.
3. **Combine:** assemble the subproblem solutions into a solution to the original problem.

5 Case study: merge sort

Input. An array $A[0], A[1], \ldots, A[n-1]$ of n numbers. (Here we are ignoring bit complexity; so imagine they are all 64-bit numbers and can be compared/rearranged in $\mathcal{O}(1)$ time.)

Goal. Rearrange the numbers in nondecreasing order.

For simplicity, assume that n is a power of two. This assumption is not essential: one may split using floors and ceilings, or conceptually pad the array.



5.1 Algorithm

1. If $n \leq 1$, return A.
2. Divide A into its left and right halves.
3. Recursively sort each half.
4. Merge the two sorted halves.

The merge step repeatedly compares the first unmerged element of each half, moves the smaller element to the output, and finally appends the unused suffix of the nonempty half.

Exercise. Show that two sorted arrays, each of length n, can be merged into one sorted array of length $2n$ in $\mathcal{O}(n)$ time.

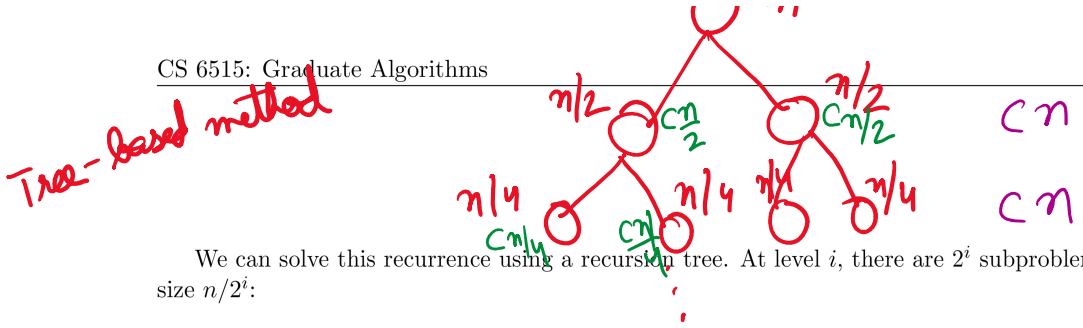
Correctness. Correctness follows by induction on n. We will discuss induction in detail in next lecture, but briefly: The base case $n \leq 1$ is immediate. For the inductive step, the recursive calls correctly sort the two halves by the inductive hypothesis. The merge procedure outputs their elements in nondecreasing order and outputs every input element exactly once. Therefore the result is a sorted permutation of the original array.

5.2 Running time

Let $T(n)$ denote the worst-case running time on an input of length n. The two recursive calls operate on inputs of size $n/2$, and the merge step takes cn time for some constant $c > 0$. Thus

$$T(n) = 2T(n/2) + cn, \qquad T(1) = \Theta(1).$$

CS 6515: Graduate Algorithms Fall 2026



We can solve this recurrence using a recursion tree. At level i, there are 2^i subproblems, each of size $n/2^i$:

Level	Number of subproblems	Merge work per subproblem	Total merge work
0	1	cn	cn
1	2	$cn/2$	cn
2	4	$cn/4$	cn
$\vdots$	$\vdots$	$\vdots$	$\vdots$
i	2^i	$cn/2^i$	cn

The subproblem size reaches 1 after $\log_2 n$ levels. Every non-leaf level does cn work, and the n leaves contribute $\Theta(n)$ total work. Hence

$$T(n) = cn \log_2 n + \Theta(n) = \Theta(n \log n).$$

Takeaway. Divide and conquer can turn a problem into many simpler copies of itself. Its efficiency depends on both the size and number of the subproblems and on the cost of combining their solutions.

Junyong Chang
Chaithanya Shyam Delanthabettu
Kevin Daniel Lewis

Bei Liu
Aidan Minh-Khang Nguyen
Aryan Dixit Patel
Jack Gardner Rose
Shashwata Roy
Joseph Souaiby
Anshul Tripathy

Intro to Grad Algorithms - CS-6515-A

Course Page

We will use Piazza for announcements, discussions, lecture notes, etc. Please always ask your questions on Piazza rather than by email.

It is your responsibility to check Piazza for assignments, announcements, and review material.

Canvas is primarily used only for grades and linking to Piazza and Gradescope.

Other Material

There is no single textbook covering all the course contents. Below are some general references, and we will be providing lecture specific references as we go:

- “Algorithms” by Dasgupta, Papadimitriou, and Vazirani (**available here**)
- “Algorithms” by Erickson. Free version at **https://jeffe.cs.illinois.edu/teaching/algorithms**
- “Algorithm Design” by Kleinberg and Tardos
- “Introduction to Algorithms” by Cormen, Leiserson, Rivest, and Stein
- “Algorithms Illuminated” by Roughgarden: videos at **http://www.algorithmsilluminated.org**

Exams

- There will be **4** exams/quizzes (including finals). For exam dates, see the schedule at the end of the Syllabus.
- The first 3 exams will take place during the usual lecture time in the usual lecture hall. The 4th exam is during the final exam week.

Grade

- The final grade will be a weighted average of **exams (75%)** and **homework (25%)**.
- The worst exam grade is dropped. If you are happy with the grade of your first 3 exams, the 4th (final) exam is effectively optional.
- The average will determine your final grade. Grade thresholds are A - 90%, B - 80%, C - 70%, D - 60%. The thresholds may change in the future, but they will only change in your favor (lowered).

Homework

There will be 6 problem sets/homeworks during the semester.

- We expect there to be ~25 problems in total (~4 problems per problem set).
- Your best 15 problems will be used for your grade. The other lowest-scoring problems will be dropped. Beyond that, there are no extensions, do-overs, etc.
- The questions will be mostly theory/proof-based problems, but there will also be the occasional programming/chatgpt assignments.
- You are encouraged to collaborate and work in groups, but you must write your own solutions. You must write the names of students you worked with on your homework.
- You may look up reference material online and in textbooks to help with your homework but you may not look up, copy, or paraphrase answers to homework questions.
- We will use Gradescope for submission and grading.
- Solutions must be properly typed pdf. Handwritten solutions will not be accepted. LaTeX (via overleaf or otherwise) is strongly recommended.

Course Policies (late-submission, re-take exam, sickness, etc)

- Homework: Your best 15 problems will be used for your grade. The other lowest-scoring problems will be dropped. Beyond that, there are no extensions, late submissions, do-overs, etc.
- Exam: If you missed one of the first 3 exams for extreme circumstances that were out of your hand (e.g., sickness), contact us ASAP and provide documentation from Dean's office. In this case, we will allow you to double-count the final exam (i.e., count it as a makeup exam).

For fairness to all students, no other exceptions to these policies will be made for individual students, except for the disability policy.

Disability policy

Students with disabilities who require accommodations for access and participation in this course should register with the Office of Disability Services (ODS). Please contact ODS here:

https://disabilityservices.gatech.edu/.

Regrade requests

Regrade requests can be made for factual errors. Regrade requests cannot be made when the grader did not understand your solution. It is your responsibility to make your answers clear for the graders to understand.

Georgia Tech Honor Code

Please read the Georgia Tech Honor Code here:

https://policylibrary.gatech.edu/student-life/academic-honor-code.

Penalties for academic misconduct include getting a zero for an assignment or for the course, see here:

https://osi.gatech.edu/process/academic-misconduct-sanctioning-guidelines

Schedule

The course is structured in 3 parts, where each part consists of 8 lectures and the 9th lecture is an exam.

Part 1 (Most students have seen most of these topics):

- Divide & Conquer
- Dynamic Prog
- Graph Algos

Part 2:

- LPs
- Convex Optimization, Gradient Descent
- Approx Algos

Part 3:

- Rand Algo
- Complexity Theory

Four Exams:

1. Sep 22 on Part 1 topics
2. Oct 27 on Part 2 topics
3. Nov 19 on Part 3 topics
4. Dec 10 cumulative final exam.

Dec 10/11